

# Chapitre 3 : Quelques types structurés

## I Généralités sur les types structurés

En OCaml, au-delà des types de base (primitifs) vus précédemment, il est indispensable de manipuler des données plus complexes. Les types structurés permettent d'assembler, de modéliser et de donner du sens à des collections de données logiquement reliées.

Nous étudierons principalement les produits (tuples, enregistrements) et les sommes (types algébriques, énumérations).

## II Produit cartésien ( $n$ -uplets)

### A Définition et syntaxe

Le type produit cartésien permet d'associer un nombre fixe d'éléments de types potentiellement différents au sein d'une structure unique appelée  $n$ -uplet (ou *tuple*).

**Définition :** Un  $n$ -uplet  $(e_1, e_2, \dots, e_n)$  possède le type produit  $t_1 * t_2 * \dots * t_n$  si chaque élément  $e_i$  possède le type  $t_i$ .

✗ **Attention** ✗ Il ne faut absolument pas confondre les  $n$ -uplets et les listes !

- Un  $n$ -uplet a une **longueur fixe**  $n$  et peut contenir des éléments de **types différents**.
- Une liste a une **longueur variable** et ne peut contenir que des éléments du **même type**.

💡 **Exemple :** Déclaration et inférence de types simples de tuples :

- `(2, 3, 4)` est de type `int * int * int`
- `(1, 3.0, "hello")` est de type `int * float * string`

Le produit cartésien est particulièrement utile pour :

1. Écrire des fonctions prenant un  $n$ -uplet en argument (à ne pas confondre avec des fonctions curryfiées à plusieurs arguments).
2. Écrire des fonctions qui renvoient plusieurs résultats groupés.

💬 **Vocabulaire :** On parle de **curryfication** lorsqu'une fonction à plusieurs arguments est transformée en une suite de fonctions à un seul argument. En réalité, en OCaml, toutes les fonctions sont curryfiées par défaut.

### B Accès aux éléments d'un $n$ -uplet

#### Méthode : Accès aux éléments d'un $n$ -uplet

Pour accéder aux éléments d'un  $n$ -uplet, on utilise la syntaxe `fst` pour le premier élément et `snd` pour le second élément (pour les 2-uplets). Pour les  $n$ -uplets avec  $n > 2$ , on utilisera du pattern matching.

💡 **Exemple :** Pour un couple (2-uplet) `(42, "hello")`, on peut accéder aux éléments de la manière suivante :

```

1 let couple = (42, "hello")
2 let premier = fst couple (* renvoie 42 *)
3 let second = snd couple (* renvoie "hello" *)

```

💡 **Exemple :** Pour un 3-uplet (1, 2.0, "test"), on peut utiliser le pattern matching :

```

1 let triplet = (1, 2.0, "test")
2 let (a, b, c) = triplet (* a = 1, b = 2.0, c = "test" *)

```

### III Listes d'association

**Définition :** Une **liste d'association** est une liste dont les éléments sont des couples (des paires de type 'a \* 'b). Elle sert généralement à modéliser un dictionnaire où le premier élément de la paire représente une clé, et le second représente la valeur associée.

💡 **Exemple :** Voici une liste d'association liant des noms d'étudiants à leurs notes :

```

1 let notes = [("Alice", 18); ("Bob", 14); ("Charlie", 16)]

```

Le type de cette expression est (string \* int) list.

📌 **Remarque :** La recherche d'un élément dans une liste d'association se fait séquentiellement de la tête vers la queue. Dans le pire des cas (si la clé n'est pas présente ou est en fin de liste), l'opération est en  $\Theta(N)$  où  $N$  est la taille de la liste.

### IV Enregistrements

Les enregistrements (*records*, on se rapproche des structures en C) sont très proches des tuples, à la différence près que chaque composant est nommé. Ces composants sont appelés des **champs**.

#### Méthode : Définition et utilisation d'un enregistrement

On déclare d'abord le type avec le mot-clé `type`, puis on instancie les valeurs.

💡 **Exemple :** On peut par exemple définir un type d'enregistrement pour représenter un étudiant avec son prénom, son nom, son âge et sa note :

```

1 (* Definition du type *)
2 type etudiant = {
3   prenom: string;
4   nom : string;
5   age : int;
6   note : float
7 }
8
9 (* Instanciation *)
10 let e1 = { prenom = "Laurent"; nom = "Cai"; age = 20; note = 7.5 }

```

Pour accéder à un champ spécifique d'un enregistrement, on utilise la notation pointée : `valeur.champ`. `e1.nom` renvoie la chaîne "Cai", `e1.prenom` renvoie la chaîne "Laurent".

## V Types algébriques

Les types algébriques (ou types variants) permettent de définir des types qui peuvent prendre plusieurs formes distinctes. On utilise le symbole vertical `|` pour lister les différents constructeurs.

### A Énumérations

Le cas le plus simple d'un type somme est l'énumération, où l'on définit un ensemble de constantes nommées (les constructeurs). Les noms des constructeurs doivent impérativement commencer par une lettre majuscule.

```
1 type jour = Lundi | Mardi | Mercredi | Jeudi | Vendredi | Samedi | Dimanche
```

**Illustration :** Le type standard `bool` de OCaml est en réalité une énumération prédéfinie que l'on pourrait modéliser par `type bool = True | False`.

**Exemple :** On peut utiliser les énumérations pour représenter des états ou des catégories. Par exemple, un type pour les directions cardinales :

```
1 type direction = Nord | Sud | Est | Ouest
2
3 let ma_direction = Nord
```

### B Sommes

Les constructeurs d'un type variant peuvent également transporter des valeurs (des arguments) grâce au mot-clé `of`.

**Exemple :** On peut utiliser les sommes pour représenter des formes géométriques avec des propriétés spécifiques :

```
1 type forme =
2   | Cercle of float (* Transporte le rayon *)
3   | Rectangle of float * float (* Transporte la largeur et la hauteur *)
```

#### Méthode : Filtrage par motif sur un type somme

Le moyen le plus efficace et le plus sûr de manipuler un type somme est d'utiliser le pattern matching, car le compilateur vérifie l'exhaustivité des cas.

**Exemple :** On peut utiliser le *pattern matching* pour calculer la surface d'une forme :

```
1 let surface f =
2   match f with
3   | Cercle r -> 3.14159 *. r *. r
4   | Rectangle (l, h) -> l *. h
```

### C Sommes polymorphes

Il est possible de rendre un type somme générique en introduisant une variable de type (notée `'a`). L'exemple le plus célèbre de la bibliothèque standard OCaml est le type `'a option`, utilisé pour représenter la possibilité d'une absence de valeur.

**Définition :** Le type `'a option` est défini par `type 'a option = None | Some of 'a`

**Exemple :** Une fonction de division sécurisée qui renvoie `None` en cas de division par zéro :

```

1 let diviser a b =
2   if b = 0 then None else Some (a / b)

```

## VI Types récursifs

Un type algébrique est dit récursif s'il se mentionne lui-même dans sa propre définition. C'est le cas typique des structures de données dynamiques comme les listes ou les arbres.

### A Définition alternative des listes

Bien que les listes soient natives en OCaml (`[]`, `::`), nous pouvons tout à fait réimplémenter notre propre type de liste polymorphe et récursive :

```

1 type 'a ma_liste =
2   | Nil
3   | Cons of 'a * 'a ma_liste

```

 **Exemple** : Représentation de la liste `[42; 17]` avec notre nouveau type :

```

1 let l1 = Cons(42, Cons(17, Nil))

```

### B Application : Implémentation de `map`

Nous pouvons maintenant réécrire une fonction d'ordre supérieur effectuant une transformation sur notre liste récursive grâce au filtrage :

```

1 let rec mon_map f l =
2   match l with
3   | Nil -> Nil
4   | Cons(h, t) -> Cons(f h, mon_map f t)

```

**Analyse de la signature** : Le compilateur infère automatiquement le type suivant :

```
val mon_map : ('a -> 'b) -> 'a ma_liste -> 'b ma_liste = <fun>
```

### C Modélisation d'expressions arithmétiques

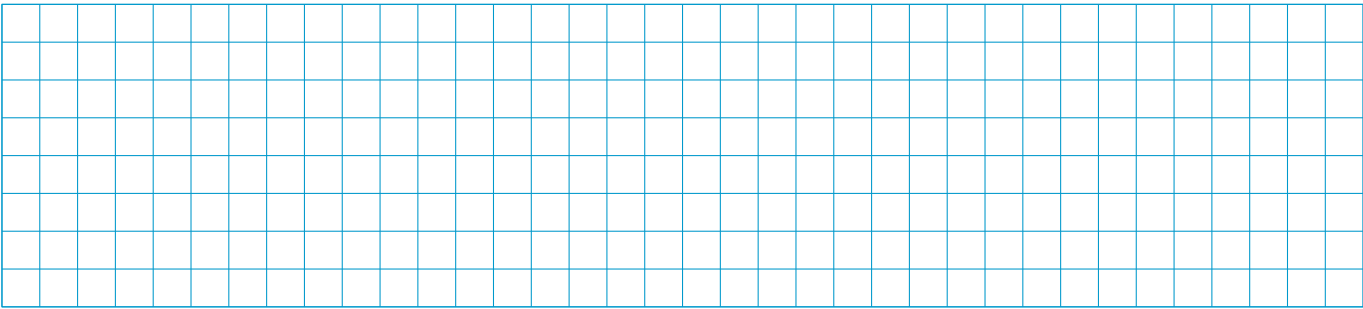
Les types récursifs se prêtent particulièrement bien à la création d'Arbres de Syntaxe Abstraite (AST) pour modéliser des langages ou des opérations mathématiques.

```

1 type expr =
2   | Var of string
3   | Const of int
4   | Add of expr * expr
5   | Mult of expr * expr

```

 **Application** : Écrire une fonction récursive `eval : (string * int) list -> expr -> int` permettant d'évaluer la valeur numérique d'une `expr` au sein d'un environnement donné (modélisé par une liste d'association contenant la valeur des variables).



## Contents

|            |                                                            |          |
|------------|------------------------------------------------------------|----------|
| <b>I</b>   | <b>Généralités sur les types structurés</b>                | <b>1</b> |
| <b>II</b>  | <b>Produit cartésien (<math>n</math>-uplets)</b>           | <b>1</b> |
| A          | Définition et syntaxe . . . . .                            | 1        |
| B          | Accès aux éléments d'un $n$ -uplet . . . . .               | 1        |
| <b>III</b> | <b>Listes d'association</b>                                | <b>2</b> |
| <b>IV</b>  | <b>Enregistrements</b>                                     | <b>2</b> |
| <b>V</b>   | <b>Types algébriques</b>                                   | <b>3</b> |
| A          | Énumérations . . . . .                                     | 3        |
| B          | Sommes . . . . .                                           | 3        |
| C          | Sommes polymorphes . . . . .                               | 3        |
| <b>VI</b>  | <b>Types récurifs</b>                                      | <b>4</b> |
| A          | Définition alternative des listes . . . . .                | 4        |
| B          | Application : Implémentation de <code>map</code> . . . . . | 4        |
| C          | Modélisation d'expressions arithmétiques . . . . .         | 4        |

## Crédits

Ce cours est mis à disposition selon les termes de la Licence Creative Commons  **Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International**. Pour voir une copie de cette licence, visitez <http://creativecommons.org/licenses/by-nc-sa/4.0/>.